

Efficient Remote KV Cache Reuse with GPU-native Video Codec

Liang Mi^{1,*}, Weijun Wang^{2,*}, Jinghan Chen¹, Ting Cao^{2,†}, Haipeng Dai^{1,†}, Yunxin Liu²

¹Nanjing University ²Institute for AI Industry Research (AIR), Tsinghua University

Abstract

Remote KV cache reuse fetches KV cache for identical contexts from remote storage, avoiding recomputation, accelerating LLM inference. While it excels in high-speed networks, its performance degrades significantly in bandwidth-limited scenarios. Recent studies address this by transmitting KV caches in compressed form, but the associated heavyweight decompression counteracts the KV reuse benefits. In this paper, we propose an *efficient and widely deployable* remote KV cache reuse solution that leverages *GPU-native video codecs*. Our system, KVCodec, enables effective KV cache coding with two techniques. The *codec-friendly tensor layout* compresses the KV cache in a highly compact video format, enabling fast transmission. The *efficient KV fetcher* orchestrates the transmission, decoding, and restoration of compressed KV caches in an efficient pipelined manner, eliminating resource contention, masking network fluctuations, and achieving minimum time-to-first-token (TTFT). We prototype KVCodec on diverse GPUs from high- to low-end. Experiments reveal that it reduces TTFT by up to 3.51× while maintaining lossless accuracy, compared to SOTA methods.

CCS Concepts

- **Computing methodologies** → **Natural language generation**;
- **Networks** → **Application layer protocols**.

Keywords

KV Cache, LLM Inference, Video Codec

ACM Reference Format:

Liang Mi, Weijun Wang, Jinghan Chen, Ting Cao, Haipeng Dai, Yunxin Liu. 2026. Efficient Remote KV Cache Reuse with GPU-native Video Codec. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3789240.3829120>

1 Introduction

With context window trending to millions of tokens, Large Language Models (LLMs) have demonstrated superiority in many modern services. By ingesting informative contexts, including system prompts [14], retrieved documents [12, 25], and interaction histories [19, 20, 73, 88], together with request queries, LLMs can generate high-quality outputs powering diverse applications, from code generation [23, 35, 60] to agentic workflows [36, 69, 78]. For

*Liang Mi and Weijun Wang contributed equally to this work. This work was done while Liang Mi interns at AIR, Tsinghua University.

†Corresponding authors: Ting Cao and Haipeng Dai.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26*, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3829120>

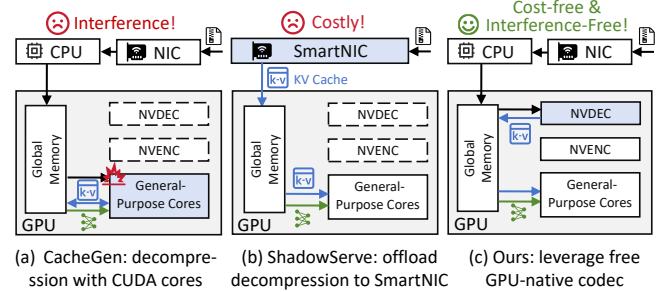


Figure 1: Solutions of remote KV cache reuse. Our KVCodec exploits GPU-native video codecs, delivering the best cost-efficiency and system performance.

efficient serving, storing the intermediate states, *KV cache*, during LLM inference has become a de facto choice.

KV cache reuse further amortizes the computational costs across multiple requests. It persists KV caches in the first inference and reuses them when future requests share common prefixes. Most LLM inference systems [1, 13, 33, 42, 43, 59, 79, 86] today integrate KV cache reuse as their key feature. Their consensus is that KV caches are frequently reused. Mooncake [59] claims that 50% of KV caches will be reused in KiMi’s real-world workloads, while LMCache [13] believes this number is higher. Despite this benefit, the storage cost of KV caches can be huge [43, 74]. Storing 80K-token KV caches (e.g., Amazon’s annual report) of a medium-level 34B model can consume up to 19GB of storage [49]. For this reason, hosting all reusable KV caches locally is impractical.

To tackle this, KV cache reuse uses external storage and fetches them on demand from remote resources. Mooncake abstracts all GPU memory, host memory, and disks within GPU clusters into a disaggregated KV cache pool, and LMCache leverages dedicated storage servers. In such distributed setups, KV caches must be transmitted over the network. Although high-speed interconnections (e.g., RDMA) render transmission latency negligible [46, 59, 64, 87], economic considerations drive modern LLM services to deploy on mid-range GPUs typically paired with only constrained bandwidths, tens of Gbps or less [26, 41, 51–53, 74].

Recent studies [49, 74] mitigate this networking bottleneck by transmitting KV caches in compressed form rather than raw tensors. It indeed yields bandwidth savings, but the associated decompression latency lies on the critical path of LLM inference, counteracting the network benefits. CacheGen [49] customizes a CUDA kernel to accelerate decompression, yet it competes for GPU resources with LLM inference engines, as the blue box shown in Fig.1(a), severely degrading LLM inference performance (§2.2). ShadowServe [74] achieves interference-free inference by offloading decompression to SmartNIC, as shown in Fig.1(b); but, the prohibitive hardware cost hinders its widespread adoption. We believe KV cache compression

is promising, but requires a cost-efficient redesign to be widely used.

Modern GPUs are typically equipped with dedicated hardware for efficient video coding. Although initially designed for media processing, their independent on-chip computing and storage units offer a great opportunity to address the above limitations (§2.3). More importantly, as dashed boxes shown in Fig.1, they are completely idle throughout the LLM inference. Motivated by this, we ask: *Can we implement a KV cache codec with GPU-native video codecs, enabling efficient remote KV cache reuse?* A concurrent study, llm.265 [77], attempts to compress KV caches by video coding, but cannot be the answer to this question. It provides very limited compression gain and lacks system co-design with inference engines (§2.4). To be the answer, we must tackle two challenges.

First, to minimize transmission delay, efficient remote KV cache reuse necessitates highly compact compression. During compression, it must map KV tensors to a video format and encode them into bitstreams using video encoders. However, this is non-trivial. A successful compression strategy must comprehensively explore and judiciously exploit both the distribution of K and V values and the characteristics of the video coding, to maximize the compression ratio without compromising LLM generation quality. Blindly representing the KV tensor as individual video frames and directly encoding them into the bitstream, like llm.265 [77], results in both suboptimal compression ratios and generation quality (§2.4).

Second, to minimize the Time to First Token, TTFT, only compact compression is not enough. It just accelerates the transmission of single video chunks, but efficient remote KV reuse still requires considerate scheduling as well as rapid and interference-free decoding and restoration*. Existing KV cache reuse systems [13, 59] hardly meet these requirements. First, their schedulers indiscriminately batch all arriving requests, leading to non-reuse requests (requests without remote KV cache reuse) being blocked by the fetching stage of requests with remote reuse. Second, their bulk KV restoration consumes excessive memory from LLM inference engines, delaying the TTFT. And last, their ill-considered design hardly copes with the networking jitter (§2.4).

In this paper, we present KVCodec, an efficient remote KV cache reuse module powered by GPU-native video codecs, to address the challenges outlined above.

To address the first challenge, we propose a *codec-friendly tensor layout* that enables a tenfold compression ratio without a drop in LLM inference accuracy. Its core idea is to skip the lossy Discrete Cosine Transform (DCT) and quantization steps of video encoding, and fully utilize the lossless intra- and inter-frame redundancy elimination capability. To achieve this, we conduct an in-depth analysis of the characteristics of video coding, the distribution of KV values, the properties of LLM architectures, and finally obtain the layout principle: slice KV tensors along the token dimension, scatter resulting tensors over continuous frames, and encode them in multiple-resolution versions (more in §3.2).

To tackle the second challenge, we propose an *efficient remote KV fetcher* (§3.3) including three key techniques. The fetching-aware scheduler (§3.3.1) discriminates the requests w./w.o KV reuses and

isolates the KV fetching in the background, preventing blocking the inference of non-reuse requests. The adaptive-resolution KV fetching (§3.3.2) adjusts the video resolution to tune the video size, enabling efficient KV fetching across varying network bandwidths. The frame-wise tensor restoration minimizes the memory cost of restoring decoded frames to original KV tensors, without impacting LLM inference engines.

We summarize our key contributions as follows:

- To the best of our knowledge, we are the first to identify the opportunity of GPU-native video codecs for remote KV cache reuse to accelerate LLM inference.
- We prototype KVCodec[†], an efficient remote KV cache reuse system powered by GPU-native video codecs. It involves two core techniques, codec-friendly tensor layout and efficient remote KV cache fetching.
- We implement KVCodec and conduct evaluations on three GPUs from high- to low-end and three different size models from 7B to 70B over 1-40Gbps bandwidths. Experimental results show that KVCodec achieves 1.52-3.51× TTFT reduction compared to SOTA methods, while ensuring high accuracy and interference-free non-reuse requests inference.

This work does not raise any ethical issues.

2 Motivation and Challenges

2.1 Preliminary of Remote KV Cache Reuse

LLM inference contains two phases. The prefilling phase processes the entire input tokens simultaneously to generate the first output token, while the decoding phase generates the subsequent ones in an autoregressive manner. For efficiency, LLM inference engines store intermediate tensors, *i.e.*, the KV caches produced by each attention layer, to eliminate the redundant computation of autoregressive generation. Although KV cache significantly speeds up decoding, it provides no acceleration for the prefilling phase.

KV cache reuse operates as a storage-for-computation trade-off, substantially reducing prefilling latency. Its core principle is that KV caches generated by one request can be reused by subsequent requests sharing a common sequence prefix. This pattern is highly prevalent across modern LLM applications. For example, to maintain consistency, chatbots [31, 32, 39, 82] feed conversation histories into the LLM at each chat round; in multi-agent code debuggers [7, 57, 78], web-search agents frequently reread buggy code from coding agents to find related posts on the Internet; and Vision-Language-Action (VLA) models [44, 48, 61] in embodied AI continuously recall prior observations and actions to boost the quality of the next action. Since this repeated content can be leveraged after the first processing, LLM inference engines [42, 86] persist their KV caches, rather than immediately freeing them, to facilitate future reuse.

Remote KV cache reuse significantly extends the capacity of reusable KV cache by retrieving them from remote resources. In large-scale LLM services [1, 29, 49, 59, 74], since the limited capacity of single node and the strategies of load-aware [22, 38] and fairness-aware [62] request dispatching, reusable KV caches are typically scattered across distant nodes. To reuse these KV

*This paper follows the same setting as prior studies [49, 59, 74], where the KV caches are chunked and compressed in advance, and stored at remote storage nodes. So remote KV reuse (a.k.a., KV fetching) consists of KV transmission, decoding, and restoration.

[†]Our code is available at <https://github.com/mi150/KVCodec>

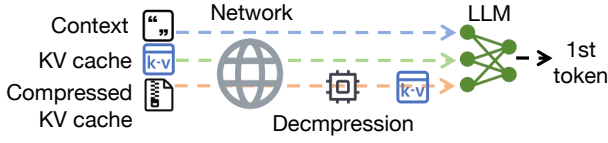


Figure 2: Current three prefilling types: full prefill, raw KV reuse, compressed KV reuse. Their time costs are: prefill, transmission+prefill, transmission+decompression+prefill.

caches in this distributed environment, LLM inference engines must fetch them from the source nodes via the network interconnect. However, remote KV cache reuse is only beneficial when the KV cache fetching speed exceeds the latency of recomputation. Early LLM serving systems [46, 59, 64] could easily achieve this benefit, as they were commonly deployed in High-Performance Computing (HPC) centers with >100Gbps RDMA interconnects. In contrast, for cost efficiency, modern serving systems [18, 21, 24, 28, 34, 52, 74] tend to be deployed on mid-range or even low-end GPUs, which communicate over bandwidth-limited networks with tens of Gbps or less. Furthermore, Kimi’s Prefill-as-a-Service [58] executes PD-disaggregation across clusters over public networks, where the bandwidth is below 100 Gbps. Some other systems [1, 26] rent dedicated storage servers to scale the capacity of reusable KV caches, yet their performance is still limited by constrained bandwidth (19 Gbps on AWS [2]).

2.2 Limitations of Existing Remote KV Cache Reuse Systems

To mitigate the network bottleneck, SOTA systems [49, 74] transmit KV caches in compressed forms. While theoretically beneficial, our empirical analysis reveals that these bandwidth savings yield only a limited end-to-end latency reduction (as elaborated shortly). To explore the reason, we benchmark the TTFT of three approaches, as shown in Fig. 2: (1) *Full Prefill* (baseline without KV reuse), (2) *Raw KV Reuse* (e.g., Mooncake [59], AIBrix [1]), and (3) *Compressed KV Reuse* (e.g., CacheGen [49], ShadowServe [74])[‡], as illustrated in Fig. 2. The evaluation is conducted on a 200k-context-window LWM-7B [80] model served by vLLM [43] on 2 NVIDIA H20 GPUs. Requests arriving follow the real-world trace [70], and network bandwidth is regulated from 1 to 40 Gbps over TCP and 100/200 Gbps over RDMA. As shown in Fig. 3, the “winning area” of the existing compressed KV reuse solution (dashed box) is surprisingly small for three reasons.

Limited TTFT gains due to suboptimal compression. Both ShadowServe and CacheGen treat KV tensors as generic byte streams and compress them with arithmetic coding, resulting in a low compression ratio. Such a compression strategy entirely ignores the unique distribution of K and V data (§3.2), rendering the reduced transmission time unable to amortize the substantial decompression delay and thus poor TTFT in most bandwidth-context-length scenarios.

[‡]These transmission-oriented methods leverage aggressive arithmetic coding to encode KV caches into bitstreams, which are orthogonal to prior KV cache compression methods (e.g., pruning [56, 84] or quantization [37, 50]) that keeps tensor formats for GPUs directly using.

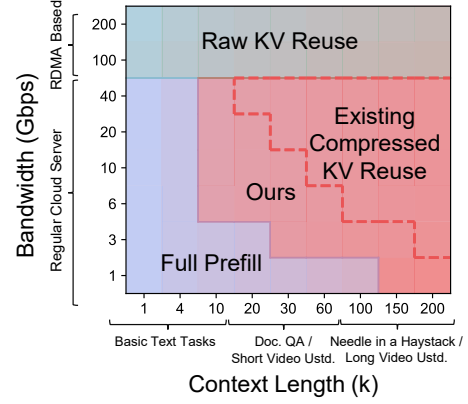


Figure 3: “Winning areas” of three prefilling types under various bandwidths and context lengths. KVCodec significantly extends the applicable scope of compressed KV reuse.

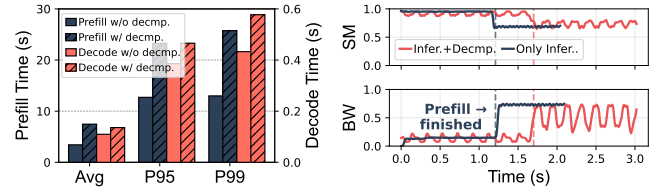


Figure 4: Concurrent LLM inference and KV decompression cause extra delay.

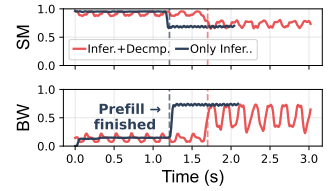


Figure 5: Kernel switch yields SM underutilization and memory I/O contention.

Severe resource contention of CUDA-based decompression.

CacheGen customizes a CUDA kernel to accelerate the decompression of KV caches. It truly decreases decompression time but causes a significant LLM inference delay, resulting in a 50% increase in prefilling time and a 20% increase in decoding time, as shown in Fig. 4. This stems from two factors. (i) *Compute preemption*: Fig. 5 profiles the Streaming Multiprocessors (SM) and GPU I/O bandwidth utilization of standalone LLM inference versus concurrent with decompression. Compared to standalone (blue curve), concurrency triggers frequent kernel context switching (the fluctuation of the red curve), leading to SM underutilization, high memory I/O contention, and thus delaying the LLM inference. (ii) *Memory bloat*: CacheGen’s decompression seizes a considerable amount of GPU memory from LLM inference engines. As shown in Fig. 6, it pre-allocates 5.5GB of GPU memory, 2.7× larger than the original KV cache, to decompress only 4K tokens. This ill-considered memory management limits the batch size of LLM inference, further increasing the average inference time.

Prohibitive deployment costs of SmartNIC-based decompression. To avoid GPU contention, ShadowServe offloads KV cache decompression to SmartNICs. It isolates the decompression from LLM inference but introduces significant deployment costs and hardware dependencies. These constraints limit the scalability and adoption of ShadowServe. In line with the current trend of deploying LLM inference systems on economical platforms, this paper aims to develop a cost-efficient and easy-deployed solution.

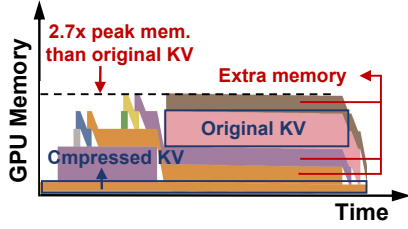


Figure 6: Peak memory of decompression in CacheGen is 2.7× that of the raw KV cache.

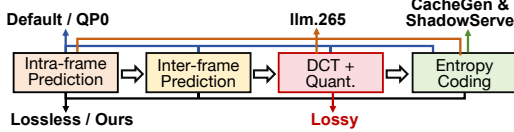


Figure 7: Standard H.265 encoding pipeline. Default and QP0 apply all steps to KV tensors while Lossless skips lossy steps. llm.265 [77] skips inter-frame prediction. CacheGen [49] and ShadowServe [74] only utilize arithmetic coding.

2.3 Opportunity of GPU-native Video Codec

Modern GPUs are typically equipped with dedicated ASIC units, e.g., NVIDIA’s NVENC/NVDEC [5], AMD’s AMF [3], and Intel’s QSV [4], for efficient video coding. Despite being designed for video, their on-chip computing and storage resources, which are totally independent of general-purpose computing units, offer us a good chance. More importantly, current LLM inference engines (e.g., vLLM [43], SGLang [86]) completely ignore these resources, leaving them fully idle during inference, as illustrated in Fig. 1.

Offloading KV cache compression and decompression to GPU-native video codecs opens up a new design space for remote KV cache reuse. It presents a valuable opportunity to address the limitations of existing methods. (i) Video codecs exploit spatial and temporal redundancies of images (tensors), offering the potential to achieve superior compression ratios beyond generic AC methods. (ii) Offloading to independent hardware isolates the workload, effectively eliminating the compute preemption and memory bloat, enabling interference-free[§] simultaneous inference and KV decompression. (iii) As ubiquitous components on modern GPUs, this ASIC provides a scalable solution with zero additional hardware cost. While llm.265 [77], a concurrent study, also attempts to compress KV caches with video coding, it fails to fully exploit the compression potential of video codecs and lacks co-design with LLM inference engines (§2.4).

2.4 Challenges of Compressed KV Streaming with GPU-native Video Codec

Building an efficient remote KV cache reuse system using GPU-native codec hardware involves two challenges.

C1: Difficult accuracy-compression tradeoff. To minimize transmission latency, the system must maximize the compression ratio while guaranteeing the inference accuracy. However, naively encoding KV tensors with video codecs results in a suboptimal tradeoff. To demonstrate this, we compare three SOTA methods CacheGen [49], ShadowServe [74], and llm.265 [77], against three

[§]Interference-free here refers to the decoding computation (NVDEC consumes zero CUDA cores), not GPU memory management (see §6).

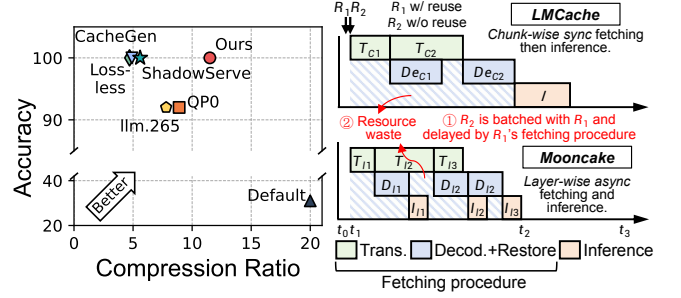


Figure 8: Current solutions yield an unsatisfactory tradeoff between accuracy and compression. Figure 9: SOTA KV management systems deliver inefficient KV between accuracy and compression. fetching on regular (non-RDMA) networks.

encoding configurations applied on KV tensors[¶] as illustrated in Fig. 7: (1) *Default* (standard NVENC settings), (2) *QP0* (set quantization parameter as zero), and (3) *Lossless* (bypassing the lossy steps).

As shown in Fig. 8, all these methods fail to balance accuracy and compression ratio. (i) *Lossy step destroys accuracy*: Default, QP0, and llm.265 achieve high compression ratios with DCT and quantization, but suffer from accuracy drops. This is because of the different sensitivity of high-frequency information between human eyes and LLM inference. The high-frequency values imperceptible to human eyes and smoothed out by DCT and quantization typically correspond to activation outliers in LLMs, which act as critical attention sinks or salient features that contain essential information for accurate inference [27, 75]. (ii) *The Lossless yields low compression ratios*: While preserving accuracy, Lossless configuration only delivers a comparable compression ratio to CacheGen and ShadowServe. This implies it fails to exploit the inter- and intra-frame compression gain^{||}, degenerating into a simple entropy coder. We attribute this failure to the naive mapping from KV tensors to video frames, which disrupts the inherent spatial-temporal redundancies (more in §3.2). Crucially, this result misled llm.265 to conclude that KV tensors lack temporal similarity and incorrectly discard the inter-frame prediction step as shown in Fig. 7.

C2: Inefficient remote KV fetching. While compact compression enables fast transmission of video chunks, efficient KV fetching remains a challenge. Naively integrating video codecs into existing remote KV reuse systems [13, 59] fails to meet this goal as three reasons. (i) *Scheduling interference (HOL Blocking)*. The schedulers of current systems naively orchestrate the fetching requests with normal ones (e.g., R_1 with R_2 in Fig. 9) in one batch. This fetching-agnostic policy severely blocks non-reuse requests from inference, increasing the TTFT. (ii) *Pipeline stalls and hardware underutilization*. As shown in Fig. 9, LMCache’s inference-blocking fetch policy leaves GPU compute resources idle until receive all KV cache. Mooncake boosts the efficiency by a layer-wise fetching-inference pipeline. But, it still wastes resources because it lacks

[¶]All three configurations first pad (nn.ZeroPad2d) KV cache in $[N, 32, 32, 128]$ shape, then reshape (torch.reshape) it to $[N, 256, 176, 3]$, i.e., N of $[256, 176, 3]$ video frames, lastly encode them into video in H.265 via NVENC.

^{||}Video encoder exploits spatial and temporal redundancy through intra- and inter-frame prediction, eliminating them when pixels can be accurately predicted from neighboring pixels or reference frames, storing only the residual between the actual pixels and the prediction.

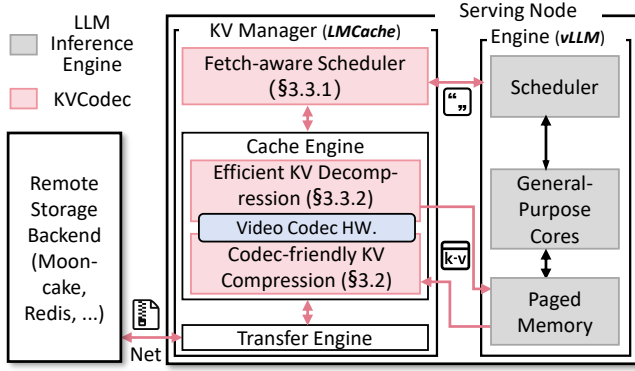


Figure 10: KVCodec overview.

mechanisms to handle the networking jitter (e.g., the longer T_{l2} in Fig.9 when bandwidth drops) and underutilizes decoding hardware (i.e., $<20\%$ NVDEC utilization under Fig.9 setting). (iii) *KV restoring memory contention*. LMCache and Mooncake restore original KV tensors from compressed form at chunk granularity. This coarse-grained design causes sudden memory spikes (e.g., 2GB per chunk, details in §3.3.2), which contend with the inference memory access, worsening the TTFT.

3 KVCodec Design

KVCodec is an efficient remote KV cache reuse solution that enables KV coding with GPU-native codec hardware by addressing the above challenges. We first provide an overview of KVCodec, then describe the core techniques it leverages.

3.1 System Overview

KVCodec integrates three modules into the original KV cache manager, as shown in Fig.10. Fetching-aware scheduler (§3.3.1) collaborates with the LLM inference engine, distinguishes the requests that need remote KV caches, and instructs the cache engine for KV fetching. KV decompression (§3.3.2), triggered by the scheduler, fetches video chunks from remote storage nodes, rapidly decodes them to video frames, restores the frames to the original KV caches, and writes them into the paged memory of the LLM engine for inference. KV compression ingests the KV tensors from the inference engine, reshapes them to a codec-friendly tensor layout (§3.2), and encodes them into videos. These KV videos are delivered to storage servers or stored in the local KV cache pool and registered as reusable. KV compression and decompression execute in the background, transparent to inference engines. KVCodec follows the same setup as prior studies [49, 59, 74], where KV caches are chunked and encoded offline.

3.2 Codec-friendly KV Compression

To minimize the transmission delay, KV caches must be compressed in a highly compact format. The typical compression gain of video encoding comes from the lossless redundant image content elimination and the lossy numerical approximation. To ensure LLM generation quality, KVCodec skips lossy steps while aiming to fully

leverage lossless redundancy elimination. To this goal, it necessitates an appropriate mapping to map the KV cache with shape $[token, layer, head_{num}, head_{dim}]$, as shown the left in Fig.13, to video format with shape $[frame, height, width, 3]$ as the right in Fig.13, allowing it to be encoded in a minimum size.

However, the simple mappings deliver very low compression ratios. For example, map KV cache to $layer/3$ number of video frames with shape $[token, channel, 3]$ (where $channel = head_{num} \times head_{dim}$) like llm.265, i.e., slice the KV cache in Fig.13 horizontally and serve every three continuous layers as one frame, and encodes them to H.265 bitstream, yielding only 58% of ours compression ratio. Similarly, slicing the KV cache along the token dimension, i.e., vertically in Fig.13, as in CacheGen, delivers only 42% of ours.

To maximize the compression ratio, we propose a *codec-friendly tensor layout* comprising two stages. In inter-frame layout (§3.2.1), it determines how to split the KV cache and where to place the resulting tensors on each frame to expose the most redundancy that video encoding can leverage. In intra-frame layout (§3.2.2), it searches for the optimal tensor shape and inner-tensor element permutation to achieve the maximum compression ratio.

3.2.1 Inter-frame layout. To guide the inter-frame layout, we first analyze the characteristics of the KV cache and video encoding, deriving three observations.

(i) *Slicing KV cache along token dimension yields the highest image similarity.* To identify the axis providing the most redundancy, we slice the KV cache along each dimension, treat sequential slices as consecutive frames, and evaluate their visual similarity with SSIM and PSNR^{**}. As shown in Fig.11, token dimension yields the highest similarity scores. We attribute this to the architectural properties of LLM. Modern LLMs' causal self-attention injects information from preceding tokens into the subsequent ones, which makes them blend with each other; and the similar positional encoding of neighboring tokens further brings them closer.

(ii) *Placing a set of tensors over multiple frames delivers more compression gain than on one single frame.* As shown in Fig.12(top), given four consecutive tensors sliced from the KV cache, serving them as four consecutive frames and encoding them into a video yields a $1.6\times$ compression gain compared to stitching them in a single frame. This benefit stems from the differential compression nature of video encoding. When stitching into a single frame, each pixel block (tensor) can only refer to boundary pixels from its adjacent left and upper blocks (e.g., the red lines in Fig.12(top)), which wastes many referable opportunities within the blocks. In contrast, when serving the tensors as four consecutive frames, the pixel block can refer to all pixels in its predecessors, resulting in a higher compression ratio.

(iii) *Compression ratio is sensitive to the video resolutions.* It seems we can easily serve each tensor as a video frame to minimize video size and transmission delay; however, such a video cannot be decoded (144P is the smallest feasible resolution for NVDEC). As shown in Fig.12(bottom), while the video size increases with higher resolution as more tensors are stitched into a single frame, the

^{**}Structural Similarity Index Measure (SSIM) and Peak Signal to Noise Ratio (PSNR) are the most widespread metrics to measure similarity of two images. The evaluation result of PSNR and visualization is detailed in Appx A.1.

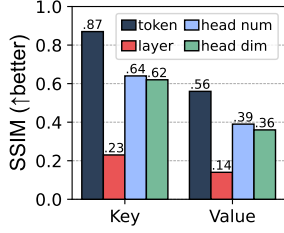


Figure 11: Similarity analysis of slicing KV cache along different dimensions.

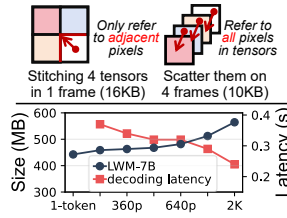


Figure 12: Tensor encoding of diverse placement (top) and resolutions (bottom).

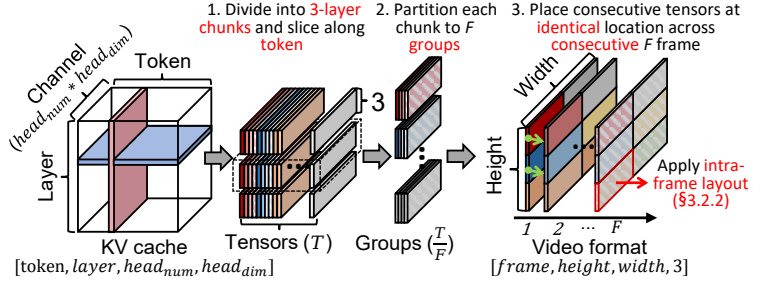


Figure 13: Inter-frame layout splits the KV cache into tensors and distributes them onto video frames to create the most temporal redundancy. The layout of each tensor is in §3.2.2.

decoding efficiency also benefits. Given this complex relationship, video resolution should be carefully set.

With these observations in mind, inter-frame layout obeys two principles. (1) Slice KV cache along the token dimension and place the token-adjacent tensors onto continuous frames (observations (i) and (ii)), constructing the maximum temporal redundancy. (2) Encode videos in multiple-resolution versions. Runtime can adaptively select the sweet-spot resolution that minimizes TTFT (observation (iii)), balancing transmission and decoding time (§3.3.2).

Following these two principles, our inter-frame layout consists of three steps as illustrated in Fig.13. 1) It first divides the KV cache into three-layer chunks and slices them along the token dimension; each chunk contains T tensors, each with shape $[1, 3, \text{channel}]$. 2) T tensors in each chunk, e.g., the dashed box in Fig.13, are sequentially partitioned into $\frac{T}{F}$ groups, each containing F tensors. 3) Adjacent tensors in each group are placed at identical positions across consecutive K frames, and the best tensor layout is determined with intra-frame layout. Such an inter-frame layout maintains the spatial alignment of consecutive token tensors, enabling the encoder to predict each tensor by referencing its temporal predecessor (green arrows in Fig.13), thereby maximizing temporal redundancy. Moreover, the three layers (lowest similarity in Fig.11) are mapped to independently coded color channels (YUV/RGB).

3.2.2 Intra-frame layout. After slicing the KV cache into three-layer token tensors and placing them in the appropriate locations, our next goal is to determine the mapping of remaining head_{num} (H) and head_{dim} (D) dimensions. Keeping the original tensor layout without adjustment, e.g., the $[1, 3, \text{head}_{\text{num}} \times \text{head}_{\text{dim}}]$ as shown the upper left in Fig.14, can only deliver an 8.7× compression ratio, much smaller than our ultimate 11.9×. Therefore, we must search for the best intra-frame layout to optimize the compression ratio.

We formulate the inter-frame layout as a joint optimization problem that couples geometric tiling (tensor reshaping) with inner-element permutation. It creates a $O(\log N \times N!)$ of solution space, $N = H \times D$, where brute-force searching is intractable. As exemplified in Fig.14, given a (1, 4096) vector of one-layer tensor, there exists 13 geometric tiling times 4096! permutation solutions. Enumerating such a huge number of possibilities and evaluating the video size of each is impossible.

To reduce the search space, we follow three rules deriving from the characteristics of LLM architecture.

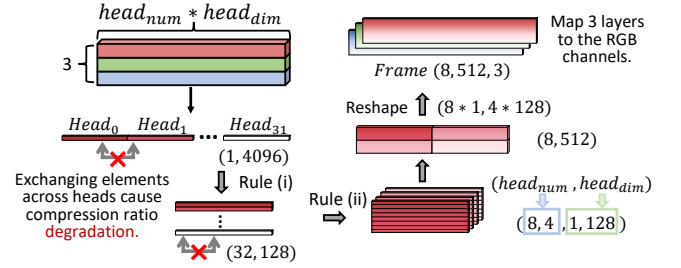


Figure 14: Intra-frame layout searches for the optimal mapping for the tensor providing the max compression ratio.

Rule (i): Do not exchange the elements from different attention heads. In LLMs, each attention head captures a unique semantic feature. Exchanging elements between one head and another disrupts this semantic information. It breaks the similarity among continuous tokens and inter-frame prediction, leading to a large residual. Our experimental results demonstrate that exchanging 50% of elements across 32 attention heads results in a 2.4× degradation in compression ratio. According to this, we isolate the element permutation across attention heads and separately consider head_{num} and head_{dim} , as (1, 4096) to (32, 128) shown in Fig.14. It eliminates enumerating all 4096! permutation, reducing the entire searching space from $O(\log N \times N!)$ to $O((\log H \times H!) \times (\log D \times D!))$.

Rule (ii): Keep the order of elements within the attention head. As the attention head serves as the fundamental semantic unit of LLMs, elements within a head jointly represent a specific feature. Disarranging the elements of the attention head disrupts its structured feature correlations, hindering intra-frame prediction. In our experiments, exchanging 50% of elements in the attention head increases the 17% size of intra-predicted frames. For this reason, we preserve the inner-head element order, which omits the elements permutation of each attention head and reduces the searching space from $O((\log H \times H!) \times (\log D \times D!))$ to $O((\log H \times H!) \times \log D)$.

Rule (iii): Arrange the order of attention heads as initial. Since distinct heads extract independent semantic features without an explicit order, their relative positions have a negligible impact on compression. Our empirical studies demonstrate that random head orders yield only <0.3% of size variation. Accordingly, we do not permute the attention heads but solely search the geometric tiling of head_{num} , which ultimately reduces the search space to very limited

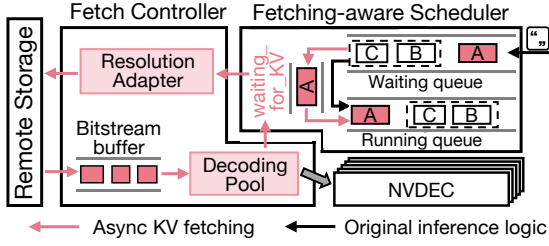


Figure 15: Control flow of KV fetching. Fetching-aware scheduler discriminates the requests. Fetch controller isolates the KV fetching procedure.

$O(\log H \times \log D)$. As shown in Fig.14, it only requires evaluating the compression ratio of $\log 32 \times \log 128 = 35$ possibilities.

Based on the principles above, the optimal layout converges to only a few dozen options. Since all these principles depend solely on the model architecture and video encoding, namely being input-agnostic (more in §5.3), we can search for the best layout offline. Spending only 1.5 hours, we obtain the best layouts (8, 512), (8, 128), and (16, 64) for all three models used in this paper, LWM-7B, Yi-34B, and Llama-70B. As exemplified in Fig.14, the $head_{num}$ and $head_{dim}$ of LWM-7B are reshaped to (8, 4) and (1, 128), and then further reshape to (8, 512). Lastly, three (8, 512) matrices from each layer are batched into a (8, 512, 3) tensor.

3.3 Efficient Remote KV Fetching

To minimize TTFT, we propose three key techniques, enabling KV-Codec efficient remote KV cache fetching.

3.3.1 Fetching-aware scheduler. Existing remote KV reuse systems batch requests without discrimination, causing requests with remote KV fetch to block the inference of non-reuse requests (§2.4). To tackle this, KV fetching must be isolated from the inference engine’s main execution flow. We propose a *fetching-aware scheduler* that enables asynchronous KV fetching, avoiding blocking non-reuse requests.

As shown in Fig.15, our scheduler imports a dedicated queue, `waiting_for_KV`, to manage fetching requests. It is located outside the LLM inference engine and collaborates with the original queues. In each iteration, the scheduler distinguishes the fetching requests, moves the eligible ones (e.g., request A) from the waiting queue into the `waiting_for_KV` queue, and notifies the fetch controller to start fetching their KV caches in the background. The non-reuse requests (e.g., B and C) still follow the original logic of LLM inference engine, entering the running queue for immediate inference. Once fetching completes, the fetch controller asks the scheduler to dequeue request A from `waiting_for_KV` queue to running for immediate execution in the next iteration. Benefiting from this design, the inference engine remains unaffected by KV fetching, maintaining its non-reuse or user-defined scheduling policy and computation.

3.3.2 Efficient KV Decompression. Remote KV fetching consists of three phases, involving multiple hardware devices. During the transmission phase, the resolution adapter in Fig.15 specifies the KV video chunk, fetches it from remote storage, and stores it in the

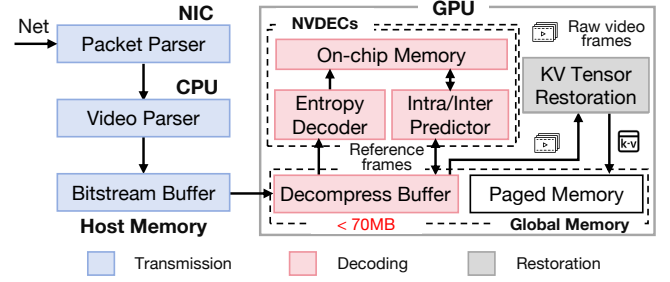


Figure 16: Dataflow of KV fetching, consisting of KV video transmission, KV video decoding, and KV tensor restoration.

bitstream buffer in host memory (see Fig.16). During the decoding phase, the decoding pool, each instance combined with an NVDEC unit, ingests video chunks and decodes them to frames. Lastly, the restoration module restores decoded frames to original KV tensors and writes them to the paged memory of LLM inference engines.

Adaptive resolution KV fetching. To minimize TTFT, we employ a *pipelining* mechanism to overlap transmission and decoding (restoration naturally overlaps with decoding, elaborated shortly). However, this is non-trivial for two reasons. First, the transmission and decoding efficiency exhibit opposite characteristics. While transmission favors low-resolution videos (e.g., 240P in observation (iii) of §3.2.1), it fails to saturate the block-parallel (64×64 pixels) decoding units [65], incurring $1.3\times$ higher decoding latency than 1080p as the table shown in Fig. 17. Second, the entire KV fetch may take dozens of seconds, during which network conditions are very likely to fluctuate. Static video chunk sizes suffer from dynamic bandwidth, leading to pipeline bubbles.

To remove these bubbles, we propose a *bandwidth-aware resolution adaptation* mechanism. Unlike CacheGen, which pursues transmission efficiency through aggressive quantization at the cost of accuracy, our approach adjusts the tensor-to-frame layout (*i.e.*, resolution) and so the video chunk size, adapting to dynamic bandwidth without accuracy loss. As modern GPUs are equipped with abundant NVDECs (*e.g.*, 5 NVDECs per A100), we abstract them into a decoding pool, as shown in Fig. 15, to simultaneously decode multiple video chunks. Once a decoding instance is idle, one chunk is dequeued from the bitstream buffer for immediate decoding.

Our adaptation mechanism uses a profile-based table-lookup method to select the optimal resolution for the video chunk to be fetched. When fetching a video chunk, the resolution adapter predicts the network bandwidth (for simplicity, calculated from the last chunk’s transmission delay), traverses the transmission latency for all resolutions, estimates decoding delay by table look up, and then selects the optimal resolution that delivers the minimum bubble (Appx.A.2 details pseudo code and complete lookup tables). To make it concrete, Fig.17 illustrates an example. Before fetching the 2nd chunk, the bandwidth degrades from 6 to 3 Gbps. Resolution adapter estimates the transmission delay for all resolutions, looks up their decoding times (as shown in the table), and selects the optimal 240p that delivers the smallest 6ms bubble. Four chunks later, bandwidth increases to 4 Gbps, so the adapter switches to 480p. Compared to the fixed 1080p approach, our adaptive-resolution method eliminates most bubbles, thereby saving 21% in time cost.

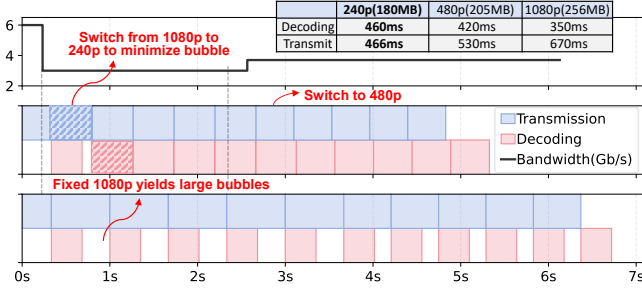


Figure 17: Adaptive resolution KV fetching significantly reduces the pipelining bubbles between transmission and decoding under dynamic bandwidth.

Frame-wise KV tensor restoration. During the decoding phase, as illustrated in Fig.16, the entropy decoder decomposes the bitstream into intra- and inter-frame parts and stores them into on-chip memory; then, the intra- and inter-frame predictor reads them out to reconstruct raw video frames. Due to the limited on-chip memory capacity, inter-frame prediction typically keeps the reference frames in GPU global memory, competing with the LLM inference engine.

To achieve interference-free decoding, we propose *frame-wise KV restoration*, which minimizes reliance on global memory via two strategies. (1) With careful tensor layout in KV compression, the number of reference frames is limited to less than four frames. This significantly reduces memory overhead to less than 20MB, even at 2K resolution. (2) Unlike CacheGen and ShadowServe that restore original KV tensors in chunks (1.5K tokens per chunk), our approach executes frame-wise. Once a raw video frame is decoded, it is immediately mapped back to the original KV tensors and filled into the preallocated slots in the paged memory. Some local cache policies [32, 82] store the original KV tensors in host memory or disks. We follow the CacheGen and Shadowserve, filling them into paged memory directly.

We implement this as a callback function, `On_frame_probe`. This delivers two-fold advantages. On the one hand, the callback function can be plugged into the codec, naturally pipelining with decoding phase. On the other hand, such a frame-wise restoration brings remarkable memory reduction to the decompress buffer, from the prior chunk-wise 1.5-2GB to the frame-wise 50MB. Consequently, combining the memory cost of reference frames, the overall decompress buffer is less than 70MB. Notably, although `On_frame_probe` executes on CUDA cores, it is super lightweight with only a few reshape operations. The frame-to-tensor mapping are on-hand encoded in the bitstreams during KV compression.

4 Implementation

We implement KVCodec as a pluggable backend of LMCACHE(v0.3.7) [13], and use vLLM(v0.10.2) [43] as the LLM inference engine. The codebase primarily uses Python (v3.12), and the efficient KV fetcher is partially implemented in C++ and CUDA, totaling ~5K LOC. (1) The KV compression is implemented with FFmpeg [8] API interface with NVENC, and it falls back to CPU encoding when lacking NVENCs. The encoding follows H.265 with `lossless=1` to skip

lossy steps. As with CacheGen, the KV cache is quantized to integers and encoded into video chunks (each containing 10K tokens across three layers) in multiple resolutions, before being registered as reusable. (2) For efficient KV fetching, the fetching-aware scheduler runs on a standalone thread integrated into LMCACHE, synchronizing with the vLLM scheduler via thread events. Adaptive resolution adaptation is implemented simply in Python, but for concurrent video decoding, all NVDECs are abstracted into a resource pool, with each instance pinned to an NVDEC for a decoding pipeline built on GStreamer [9] via `nv412decoder`. The decoding pipeline is implemented in C++ and invoked by Pybind11 with `py::gil_scoped_release` to bypass the Python GIL, ensuring that decoding does not block the vLLM main loop, and frame-wise KV tensor restoration is implemented as a `On_frame_probe` callback plugged in it, with a customized `Sparse_frame_KV_transfer` operator fast write KV tensors into paged memory. When multiple fetching requests are routed to a single serving node, KVCodec applies a FCFS policy if a request consumes the entire bandwidth or GPU memory; otherwise, it batches concurrent requests and partitions bandwidth evenly as CacheGen did for their KV fetching.

Compatibility. Since the seamless integration between LMCACHE and vLLM, KVCodec maintains full compatibility with vLLM’s native features. Notably, unlike LMCACHE’s blocking inference for fetching, KVCodec follows Mooncake’s layer-wise fetching-inference pipelining. It immediately enqueues fetching requests into vLLM’s running queue, provided the remaining layers’ fetching time can be hidden by inference. Thanks to the inherent chunked prefill from vLLM, the inference time cost can be precisely estimated, hence eliminating the pipelining bubbles of this layer-wise design. For more details, please see Appdx.A.3.

5 Evaluation

We evaluate KVCodec on three heterogeneous GPU clusters with real-world request traces. The key takeaways are:

- KVCodec reduces up to 3.51× TTFT for fetching requests and 77% TTFT and 35.4% TPOT for non-reuse requests. The performance gains are consistent across diverse models, GPUs, and network bandwidths. (§5.2)
- Codec-friendly KV compression and adaptive-resolution KV fetching bring remarkable bandwidth and TTFT savings, and frame-wise KV tensor restoration guarantees the interference-free inference. (§5.3)

5.1 Experimental Setup

Models. We evaluate KVCodec on three models of different sizes and context capabilities: LWM-7B [10] with 1M, Yi-34B [6] with 200K, and Llama3-70B [11] with 128K.

Datasets. We evaluate KVCodec on three long-context benchmarks to comprehensively assess performance across various tasks and context lengths. (1) *L-Eval* [16] contains 20 closed-ended QA tasks, including 508 long documents with lengths ranging from 3-200K tokens. (2) *LV-Eval* [83] further increases difficulty and mitigates knowledge leakage, consists of single-hop and multi-hop QA of distraction documents, fact confusion, and keyword replacement with lengths ranging from 16-256K. (3) *LongBench-V2* [17] is the most systematic evaluation that employs a multiple-choice

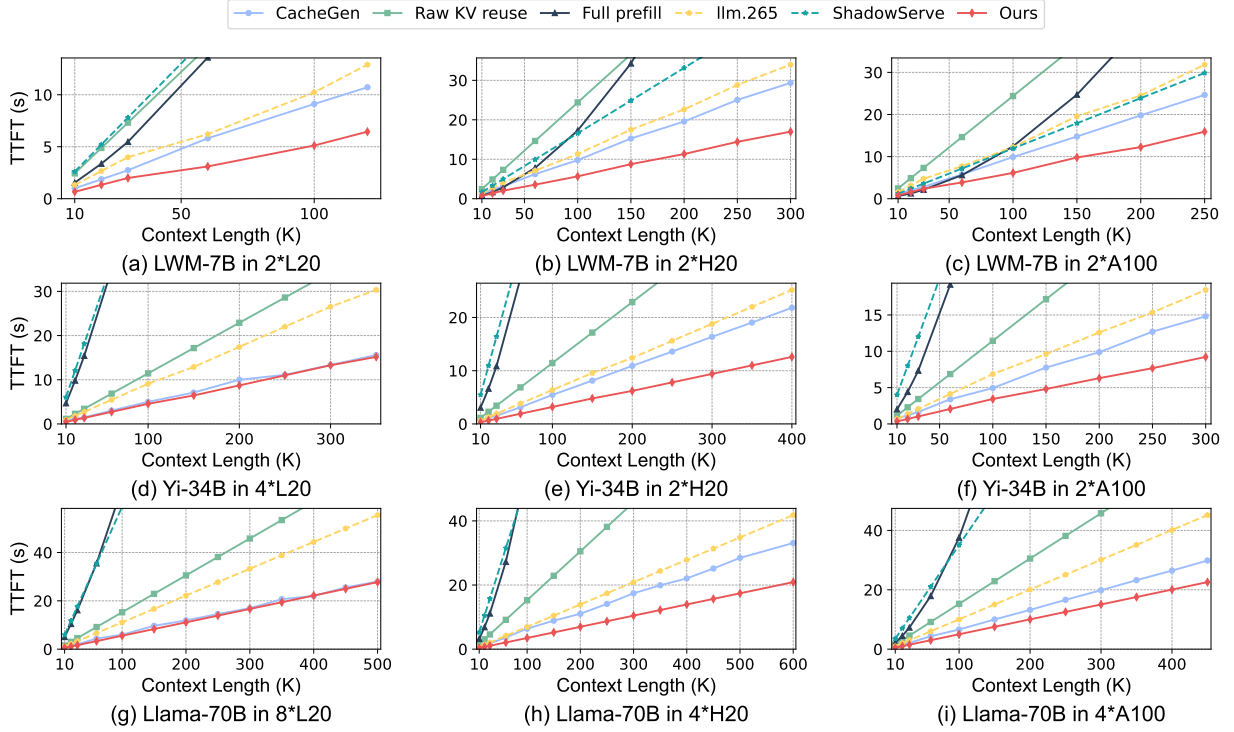


Figure 18: TTFT of the request with remote KV reuse across different context lengths over various devices and models.

format to ensure objectivity across single- and multi-document QA, long-form conversations, coding, and structured data tasks, with 13-167K context lengths.

Metrics. We compare KVCodec’s *accuracy*, *latency*, and *compression ratio* with baselines. Accuracy follows the standard metric of each dataset. For L-Eval and LongBench, it is the percentage of generated answers that correctly match the ground truth; for LV-Eval, it is the F1 score, harmonic mean of precision and recall, between the generated answer and the ground-truth answer for the QA task. Latency and compression ratio still follow §2.4.

Baselines. To show the superiority brought by KVCodec, we compare it with: *Full prefill* executes the standard LLM inference without any KV cache reuse; *Raw KV reuse* pulls the raw KV cache remotely and computes their cross attention with query prompts; and three Compressed KV reuse methods, *CacheGen*, *ShadowServe*, and *llm.265*. Furthermore, we also compare with two concurrent KV quantization methods, *KIVI* [50] and *KVTC* [63], which apply low-bit quantization and dimensionality reduction to shrink KV cache size.

Test platform. We test on three GPUs representing different market positions, each with a different number of cards, supporting different model sizes. For high-end NVIDIA A100 (80GB) with 5 NVDECs and mid-range NVIDIA H20 (96GB) with 7 NVDECs, LWM, Yi, and Llama3 leverage 2, 2, and 4 cards, respectively. For low-end NVIDIA L20 (48GB) with 3 NVDECs, three models utilize 2, 4, and 8 cards.

5.2 End-to-End Performance

This section reports the E2E performance of KVCodec on various devices. If not specified, all results are tested on H20 with the Yi-34B

model over a 16Gbps bandwidth network, which is typically offered by regular cloud platforms [74].

TTFT saving of requests requiring KV fetching. As illustrated in Fig. 18, KVCodec achieves the lowest TTFT for the fetching request across various hardware and models with varying context lengths. It outperforms Full prefill, Raw KV reuse, and CacheGen by 13.63×, 3.51×, and 1.52× on average, and is 2× faster than llm.265 and 8× faster than ShadowServe. Full prefill is extremely inappropriate for long-context LLM serving; the superlinear computational complexity with context lengths causes unacceptable TTFT. Although raw KV reuse and CacheGen reuse remote KV caches to mitigate this heavy computation, their TTFTs are bottlenecked by transmission over regular cloud bandwidth because they lack efficient KV compression. For llm.265, we replace KVCodec’s codec-friendly layout with its layout-agnostic H.265 encoding and without efficient remote KV fetching. llm.265 is bottlenecked by its low compression ratio and inefficient KV fetching: its layout-agnostic encoding forgoes the temporal redundancy that KVCodec’s codec-friendly layout exploits. For ShadowServe, we offload its decompression to the host CPU in place of the unavailable SmartNIC. ShadowServe relies on a SmartNIC to absorb its decompression cost; lacking the SmartNIC, emulating that decompression on the CPU is far too slow and dominates TTFT. In contrast, KVCodec achieves highly compact KV compression with codec-friendly layout and efficient adaptive-resolution fetching, thereby significantly reducing transmission latency. The most limited speedup, as shown in Fig. 18 (d) and (g), occurs at serving on the L20 GPUs. This phenomenon arises because L20 has only three NVDECs, which incur a queue up for decoding, and the Grouped Query Attention (GQA) used by Yi and Llama generates a relatively smaller KV cache, which

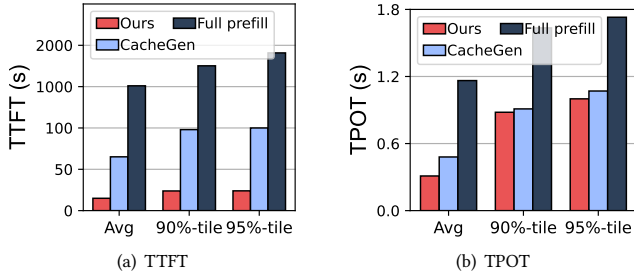


Figure 19: KVCodec’s KV fetching provides remarkable TTFT gain to non-reuse requests. KVCodec also offers TPOT benefit under PD-aggregated environments.

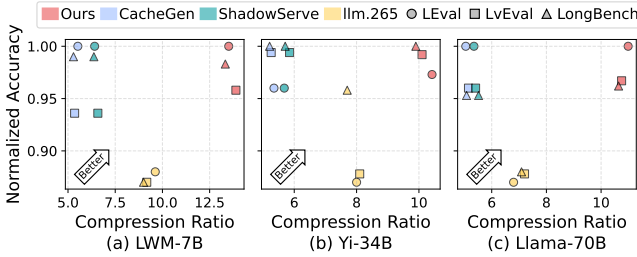


Figure 20: KVCodec achieves the best accuracy and compression ratio tradeoff over diverse models and benchmarks.

reduces the benefits of our compression. Even so, KVCodec saves remarkable TTFT for fetching requests.

TTFT & TPOT benefits for non-reuse requests. Beyond TTFT reduction for fetching requests, KVCodec keeps almost interference-free performance for non-reuse requests. We evaluate this on a real-world request trace [59] with request arrival rate at 0.2 req/s, and set 40K tokens as the reuse threshold for KVCodec and CacheGen, *i.e.*, prefill requests with <40K context tokens and reuse remote KV for >40K-token requests. All requests follow the vLLM’s default First-Come-First-Served (FCFS) policy. As illustrated in Fig. 19(a), KVCodec reduces 77.1% and 98% of TTFT compared to CacheGen and Full prefill. This stems from a two-fold. For non-reuse requests orchestrated with fetching requests in a single batch, KVCodec’s fetching-aware scheduler enables KV fetching to run in the background, avoiding blocking the inference of non-reuse requests such as CacheGen and thereby reducing their TTFT. For non-reuse requests that arrive but are still in the waiting state, all techniques in KVCodec work together, boosting the serving efficiency of running requests, reducing the waiting time for all requests, and thus saving significant TTFT. KVCodec also reduce TPOT by 35.4% and 40% compared to CacheGen and Full prefill. Full prefill must compute the input contexts for all requests. Such a large amount of computing workload means vLLM must always piggyback decoding on prefiling requests, severely delaying their TPOT. Remote KV reuse in KVCodec and CacheGen eliminates the prefiling workload, allowing decoding to run in isolation and resulting in low TPOTs. Moreover, compared to CacheGen’s CUDA-based decompression, which competes for GPU resources, KVCodec’s codec-based method provides better performance.

High compression ratio without accuracy drop. As shown in Fig.20, KVCodec achieves the best compression ratio across all

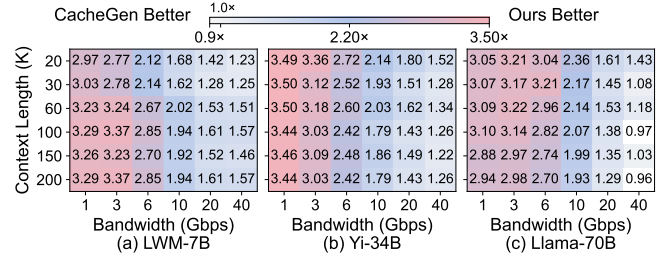


Figure 21: Performance comparison between CacheGen and ours. The value is the ratio of CacheGen’s TTFT ÷ our TTFT.

Node configuration	Break-even bandwidth
8×L20	~80 Gbps
8×A100	~200 Gbps
8×H20	~280 Gbps

Table 1: KVCodec offers better TTFT provided the network bandwidth is below the break-even thresholds on the given 8-GPU node.

datasets and models while maintaining lossless accuracy^{††}. It improves the compression ratio by 2.17× over CacheGen and 1.93× over ShadowServe, without accuracy degradation; and against llm.265, KVCodec delivers a 12% accuracy enhancement alongside 1.41× of compression ratio. The preservation of accuracy is achieved by strictly using the H.265 lossless mode, which bypasses the lossy steps of video encoding, and the same quantization method as CacheGen and ShadowServe. The superior compression performance stems from a codec-friendly tensor layout that appropriately maps KV tensors to pixel blocks, thereby maximizing data redundancy that can be eliminated by video coding.

TTFT comparison of KVCodec v.s. CacheGen. KVCodec delivers robust performance across a wide range of bandwidth and context lengths. We conduct a comprehensive evaluation of TTFT against CacheGen across 1-40Gbps bandwidth and 20K-200K context lengths. Fig. 21 indicates that under bandwidth constraints of <40 Gbps, KVCodec achieves an average speedup of 1.29×–3.50× over CacheGen. The performance gain diminishes as the arrival rate of the fetched video chunk exceeds NVDEC’s decoding capacity, breaking the transmission-decoding pipeline and causing waiting (more in §5.3). Conversely, CacheGen’s CUDA-based decompression kernel can leverage all CUDA cores on GPUs to accelerate decompression; however, this significantly impacts the inference of non-reuse requests, as shown in Fig.19. Nevertheless, in almost all scenarios, typically under low-end GPU-paired bandwidth, KVCodec still beats CacheGen thanks to our careful design.

Break-even bandwidth and decoding-pool sufficiency. Despite NVDEC’s limited per-chip decoding throughput, it is not the bottleneck for TTFT. By aggregating and abstracting the NVDECs within a single node through our decoding pool, the aggregate decoding throughput exceeds the network transmission rate. We quantify this with the break-even bandwidth, beyond which raw transmission becomes faster than compressed reuse, defined by the speedup $\frac{1}{1/CR+BW/DT_{total}} \geq 1$, where CR is the compression ratio, BW the

^{††}“Lossless” refers to lossless video compression on top of the INT8 quantization shared with CacheGen and ShadowServe.

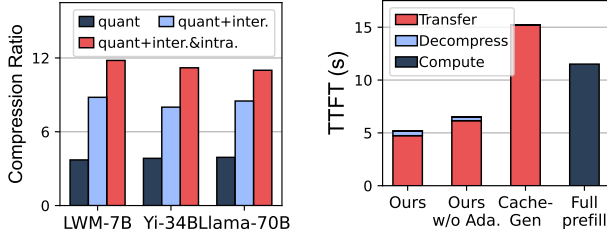


Figure 22: Compression ratio breakdown on three models.

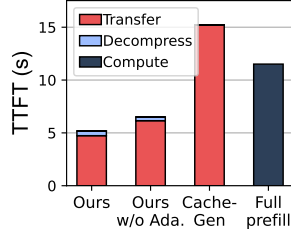


Figure 23: TTFT breakdown across different baselines.

bandwidth, and DT_{total} the aggregate decoding throughput. As shown in Tab. 1, on Yi-34B ($CR=12.5\times$), the break-even bandwidths for 8-GPU nodes reach ~ 80 , ~ 200 , and ~ 280 Gbps for L20, A100, and H20, respectively. KVCodec therefore benefits all Ethernet bandwidths below these thresholds; even the $8\times L20$ node, which has the fewest NVDECs, comfortably covers typical Ethernet deployments (<100 Gbps).

5.3 Ablation Study

We evaluate the effectiveness of each component in KVCodec by ablation studies.

Contribution of each component. Each component of KVCodec contributes to the overall performance improvement. Under the default setting ($2\times A100$, Yi-34B, 16 Gbps), removing the codec-friendly layout inflates the video size and slows TTFT by $1.6\times$, while removing the fetching-aware scheduling orchestrator slows TTFT by $1.3\times$ due to pipeline stalls and scheduling interference. The two components provide complementary gains, confirming that both compact compression and efficient fetching are indispensable for the minimum TTFT.

Codec-friendly KV compression helps KVCodec achieve $11.9\times$ compression ratio while keeping high accuracy. Fig. 22 breaks down the compression contribution of quantization, inter-frame layout, and intra-frame layout on all three models. Compared to quantization, our inter-frame layout achieves an average compression gain of $2.2\times$, while the intra-frame layout further boosts this improvement to $2.96\times$. Furthermore, compared to LWM-7B and Yi-34B, the proportion of intra-frame layout in Llama-70B is the largest due to its fewest attention heads and usage of GQA.

Adaptive resolution KV fetching. As shown in Fig. 23, under the same bandwidth state as Fig. 17, KVCodec reduces the TTFT to 5.2s, representing a 20% improvement over the baseline without adaptive resolution. This flexibility allows KVCodec to handle network jitter without compromising accuracy, unlike CacheGen’s adaptive quantization level. Furthermore, pipelining transmission and decoding effectively hides the decoding overhead, resulting in less than 400ms latency per video chunk. And remote KV reuse reduces prefill computation to under 50ms.

Frame-wise KV tensor restoration. As shown in Fig. 24, concurrently decoding and restoring KV caches of 7 video chunks only occupies 400MB of peak GPU memory. Such marginal overhead does not interfere with vLLM, which ensures memory safety in long-context scenarios. In particular, a single KV fetching only needs NVDEC to pre-allocate 40MB for video decoding, and the restoration to reshape and dequantize with 47MB.

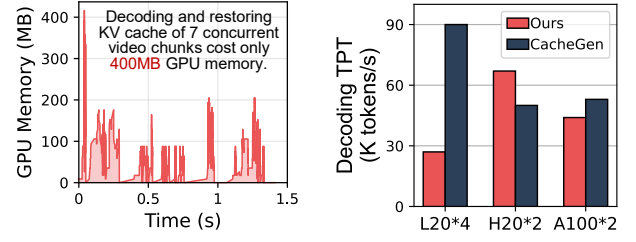


Figure 24: GPU memory cost of Figure 25: Decoding throughput concurrent 7 video chunks. on different devices.

Method	Compression	TTFT (120K)
KVCodec	12.5$\times$	$\sim 6s$
KIVI	5.3 $\times$	$\sim 8s$
KVTC	11.4 $\times$	$\sim 400s$
KIVI+KVCodec	9.56 $\times$	–
KVTC+KVCodec	16.1 $\times$	–

Table 2: KVCodec can enhance other KV quantization methods when applying KVCodec onto them.

Decoding throughput is typically bottlenecked by available NVDEC chips. We evaluate this with Yi-34B model on different platforms. As shown in Fig. 25, KVCodec achieves 27K, 67K, and 47K tokens/s on 4 L20, 2 H20, and 2 A100 platforms. They are only $0.3\times$, $1.34\times$, and $0.88\times$ the performance of CacheGen. These unsatisfactory results stem from the hardware specifications: L20 has only 3 NVDECs, while H20 and A100 have 7 and 5. In contrast, CacheGen relies on CUDA cores, which can leverage the entire GPU computing resources. Despite unsatisfactory, KVCodec’s overall performance is much better than CUDA-base CacheGen. We believe that as researchers recognize the universality of codec ASICs, next-generation GPUs will overcome these resource limitations.

Comparison with concurrent KV quantization. KVCodec delivers a better compression ratio and TTFT than the concurrent KV quantization methods, KIVI [50] and KVTC [63], and is further orthogonal and combinable with them. KIVI applies low-bit quantization to shrink tensor precision, while KVTC combines PCA dimensionality reduction with quantization and entropy coding. KVCodec optimizes the codec, while KIVI and KVTC target quantization; they are thus complementary. As shown in Tab. 2 on LV-Eval with LWM-7B, KVCodec achieves a $12.5\times$ compression ratio with $\sim 6s$ TTFT, surpassing KIVI ($5.3\times$, $\sim 8s$) and KVTC ($11.4\times$ under its best open-source config, but $\sim 400s$ due to heavy PCA-inverse decoding). Moreover, combining them is straightforward: KIVI+KVCodec encodes KIVI-quantized tensors via our codec-friendly layout and H.265, and KVTC+KVCodec replaces KVTC’s entropy coding with our layout and H.265 after its PCA and quantization. These combinations further boost the compression ratio to $9.56\times$ and $16.1\times$ compared to original, respectively.

6 Limitation and discussion

Online KV compression is essential for P-D disaggregation [87], where KV cache must be transmitted between disaggregated pre-filling and decoding nodes, and for fault tolerance [30, 66, 68] or instance preemption [53], that migrates KV cache across nodes.

While KVCodec enables compact KV compression, limited NVENC resources make the KV compression procedure insufficient to meet runtime requirements. We further quantify the feasibility of online KV compression for runtime scenarios such as P-D disaggregation and KV cache migration. On GPUs without NVENC (e.g., A100, H20), online compression avoids CUDA contention but suffers from prohibitive CPU encoding latency (~ 3 Gbps), making it infeasible. However, on next-generation GPUs (e.g., B200 with 4 NVENCs per GPU), an 8-GPU node reaches ~ 120 Gbps encoding throughput, exceeding both the KV cache generation rate (PrefaaS [59]: < 60 Gbps) and typical Ethernet bandwidth. We believe that compressed KV reuse is promising and will support a broader range of runtime scenarios with the next-generation, powerful NVENC released.

Preallocate GPU memory for fetching requests. Aligning with Mooncake and LMCACHE, KVCodec treats fetching requests as first-class citizens, preallocating memory for all KV caches upfront. Although this mechanism likely blocks the inference of non-reuse requests due to the memory budget, it ensures the timely inference of fetching requests. One feasible improvement is to store the fetched KV cache in local storage [40, 55, 76], making room for non-reuse request inference and swapping them to GPU memory when needed. We leave this for future work.

Privacy risks of shared codec hardware. In multi-tenant settings, KVCodec shares the GPU video codec hardware across co-located tenants, which may open timing or contention side channels [54], e.g., an adversary inferring the victim’s context length or fetching pattern from codec occupancy. This risk is shared by any work that repurposes shared GPU hardware, and can be mitigated by per-tenant NVDEC partitioning or constant-rate fetching. We leave a thorough security analysis for future work.

7 Related Work

LLM inference engines today employ a bunch of techniques to boost inference performance. Continuous batching [72, 81] avoids ineffective zero padding by fine-grained orchestrating prefilling and decoding workloads together for higher efficiency. Chunked prefill [15, 71] further improves it to prefill long context in chunks, mitigating execution bubbles, while prefill-decoding disaggregation [87] isolates two inference phases, eliminating interference. The Prefill-as-a-Service (PaaS) paradigm [59] further separates prefill and decoding workloads onto specialized hardware across data centers to optimize cost-performance tradeoffs. Paged attention [43] leverages paged management to eliminate memory fragmentation, while prefix caching [47, 86] reuses KV cache across requests to further reduce prefilling cost.

KV cache management becomes increasingly critical as the context length increases. Beyond the above de facto paged memory, many studies [31, 32, 39, 82] store KV cache outside the GPU, leveraging host memory and disks. They swap the inactive KV cache out and swap it back into the GPU when needed. Some other methods [67, 75, 84] are much more aggressive, identifying the cold-spot KV cache and directly evicting it permanently. KVCodec is orthogonal to these methods that require maintaining tensor formats for GPU direct use, but encodes KV caches into bitstreams to enable efficient cross-node transmission.

KV cache compression aims to reduce memory footprint while preserving model accuracy. Quantization-based methods apply low-bit representations to compress KV tensors: KIVI [50] proposes asymmetric 2-bit quantization that preserves per-channel scales, while Atom [85] employs group-wise quantization with fine-grained calibration to minimize accuracy loss. Beyond bit-width reduction, some approaches [63] explore dimensionality reduction via PCA or entropy coding to exploit redundancy in activation patterns. CacheGen [49] combines quantization with structured pruning to achieve higher compression ratios. These methods focus on in-GPU compression that maintains direct tensor access for attention computation. In contrast, KVCodec encodes KV cache into codec-native bitstreams, targeting efficient cross-node transmission rather than in-place GPU memory reduction.

Distributed KV cache management further extends caching capabilities across distributed nodes, to overcome local memory constraints. Beyond the systems mentioned in the background, some work [30, 38, 46] proposes granular paging and prefetching mechanisms to speed up or mask retrieval delays. To tackle the transmission issue, some architectural works [45, 59] explore disaggregated memory pooling via RDMA, uses zero-copy to bypass CPU during data transfer. However, pioneering providers such as FireworksAI, Baidu, and Cloudflare increasingly deploy LLM serving on low-end GPUs without RDMA for cost-effective. So other work [49, 74] compresses the KV cache before transmission, whereas KVCodec leverages hardware video codecs to enable efficient remote reuse of the KV cache.

8 Conclusion

In this paper, we study the remote KV cache reuse for LLM inference. We find that existing methods provide only suboptimal KV compression and their decompression either competes for inference resources or requires new hardware devices at extra cost. Unlike them, we leverage idle GPU-native chips and propose KVCodec system. KVCodec contains a codec-friendly tensor layout that compress KV caches to compact video formats, and an efficient remote KV fetcher that hides network latency and eliminates resource contention. In our evaluation across heterogeneous models and hardware, KVCodec delivers significant speedups in TTFT compared to SOTA methods while maintaining high accuracy.

9 Acknowledgment

We would like to thank the anonymous reviewers and shepherd, for their valuable guidance. Ting Cao and Haipeng Dai are the corresponding authors of this paper. This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB4502400, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant No. JYB2025XDXM901, in part by the National Natural Science Foundation of China under No. Grant 62272223, U22A2031, 62402280, in part by the CPSF (No.2024M761683), in part by the Tsinghua University (AIR)-AsiaInfo Technologies (China), Inc. Joint Research Center for 6G Network and Intelligent Computing (20233910006), Xiongan AI Institute.

References

- [1] Aibrix: An open-source, large-scale llm inference infrastructure for system research. <https://github.com/vllm-project/aibrix>.
- [2] Amazon ec2 instance types. https://aws.amazon.com/ec2/instance-types/?nc1=h_ls.
- [3] Amd advanced media framework. <https://gpuopen.com/advanced-media-framework/>.
- [4] Intel quick sync video installation. <https://www.intel.com/content/www/us/en/architecture-and-technology/quick-sync-video/quick-sync-video-installation.html?wapkw=quick%20sync%20video>.
- [5] Nvidia video codec sdk. <https://developer.nvidia.com/video-codec-sdk>.
- [6] 01-ai/yi-34b. <https://huggingface.co/01-ai/Yi-34B>, (Accessed on 02/04/2026).
- [7] Cursor - the ai code editor. <https://cursor.com>, (Accessed on 02/04/2026).
- [8] Ffmpeg, a complete, cross-platform solution to record, convert and stream audio and video. <https://www.ffmpeg.org/>, (Accessed on 02/04/2026).
- [9] Gstreamer: open source multimedia framework. <https://gstreamer.freedesktop.org/>, (Accessed on 02/04/2026).
- [10] Largeworldmodel/lwm-text-chat-1m. <https://huggingface.co/LargeWorldModel/LWM-Text-Chat-1M>, (Accessed on 02/04/2026).
- [11] meta-llama/llama-3.3-70b-instruct. <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>, (Accessed on 02/04/2026).
- [12] Introducing claude 4. <https://www.anthropic.com/news/claude-4>, (Accessed on 07/14/2025).
- [13] Lmcache. <https://github.com/LMCache/LMCache>, (Accessed on 07/14/2025).
- [14] Long system prompts in claude. <https://docs.anthropic.com/en/release-notes/system-prompts#august-5-2025>, (Accessed on 08/05/2025).
- [15] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
- [16] Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. L-eval: Instituting standardized evaluation for long context language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14388–14411, 2024.
- [17] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazhen Xu, Lei Hou, Yuxiao Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664, 2025.
- [18] Baidu. Baidu ai cloud: Llm inference service. <https://cloud.baidu.com/>, 2025. Accessed: 2026-06-26.
- [19] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [20] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [21] Shiye Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. Moe-lightning: High-throughput moe inference on memory-constrained gpus. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ACM ASPLOS)*, 2025.
- [22] Shiye Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in llm serving. *arXiv preprint arXiv:2501.14312*, 2025.
- [23] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [24] Cloudflare. Cloudflare workers ai: Serverless gpu inference. <https://www.cloudflare.com/developer-platform/products/workers-ai/>, 2025. Accessed: 2026-06-26.
- [25] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- [26] NVIDIA Corporation. Dynamo distributed kv cache manager (nvidia dynamo sdk v0.2.0). In *NVIDIA Corporation*, 2025.
- [27] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems (NeurIPS)*, 2022.
- [28] Fireworks AI. Fireworks ai: Generative ai inference platform. <https://fireworks.ai/>, 2025. Accessed: 2026-06-26.
- [29] Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.
- [30] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, 2024.
- [31] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [32] Shiwei Gao, Youmin Chen, and Jiwei Shu. Fast state restoration in llm serving with hcache. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 128–143, 2025.
- [33] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.
- [34] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. M\`elange: Cost efficient large language model serving by exploiting gpu heterogeneity. *arXiv preprint arXiv:2404.14527*, 2024.
- [35] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.
- [36] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- [37] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:1270–1303, 2024.
- [38] Chunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.
- [39] Jinwoo Jeong and Jeongseob Ahn. Accelerating llm serving for multi-turn dialogues with efficient resource management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1–15, 2025.
- [40] Jiazi Jiang, Yao Chen, Zining Zhang, Bingsheng He, Pingyi Luo, Mian Lu, Yuqiang Chen, Hongbing Zhang, Jiangsu Du, Dan Huang, et al. Efficient kv cache spillover management on memory-constrained gpu for llm inference. *IEEE Transactions on Parallel and Distributed Systems*, 37(1):90–105, 2025.
- [41] YOUHE JIANG, Fangcheng Fu, Xiaozhe Yao, Guoliang HE, Xupeng Miao, Ana Klimovic, Bin CUI, Binhang Yuan, and Eiko Yoneki. Demystifying cost-efficiency in llm serving over heterogeneous gpus. In *International Conference on Machine Learning (ICML)*, 2025.
- [42] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. Rgcache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- [43] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *29th Symposium on Operating Systems Principles (ACM SOSOP)*, 2023.
- [44] Mingcong Lei, Honghao Cai, Zezhou Cui, Liangchen Tan, Junkun Hong, Gehan Hu, Shuangyu Zhu, Yimou Wu, Shaohan Jiang, Ge Wang, et al. Robomemory: A brain-inspired multi-memory agentic framework for lifelong learning in physical embodied systems. In *NeurIPS 2025 Workshop on Space in Vision, Language, and Embodied AI*, 2025.
- [45] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. Rdma point-to-point communication for llm systems. *arXiv preprint arXiv:2510.27656*, 2025.
- [46] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, et al. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*, 2024.
- [47] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, 2024.
- [48] Fanqi Lin, Ruiqian Nai, Yingdong Hu, Jiacheng You, Junming Zhao, and Yang Gao. Onetwovla: A unified vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2505.11917*, 2025.
- [49] Yuhang Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.

- [50] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- [51] Ziming Mao, Tian Xia, Zhanhao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. Skyserve: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 159–175, 2025.
- [52] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ACM ASPLOS)*, 2025.
- [53] Xupeng Miao, Chunhan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotservice: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1112–1127, 2024.
- [54] Yuanqing Miao, Yingting Zhang, Dinghao Wu, Danfeng Zhang, Gang Tan, Rui Zhang, and Mahmut Taylan Kandemir. Veiled pathways: Investigating covert and side channels within gpu uncore. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1169–1183, 2024.
- [55] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. Instinfer: In-storage attention offloading for cost-effective long-context llm inference. *arXiv preprint arXiv:2409.04992*, 2024.
- [56] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. Llm1gua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *ACL (Findings)*, 2024.
- [57] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 6(3):1, 2023.
- [58] Ruoyu Qin, Weiran He, Yaoyu Wang, Zheming Li, Xinran Xu, Yongwei Wu, Weimin Zheng, and Mingxing Zhang. Prefill-as-a-service: Kvcache of next-generation models could go cross-datacenter. *arXiv preprint arXiv:2604.15039*, 2026.
- [59] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025. USENIX Association.
- [60] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [61] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryv1a: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.
- [62] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for llm serving. *arXiv preprint arXiv:2407.00023*, 2024.
- [63] Konrad Staniszewski and Adrian Łańcucki. Kv-cache tensor compression via joint decorrelation, quantization, and entropy coding. In *International Conference on Learning Representations (ICLR)*, 2026.
- [64] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [65] Gary J Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (hevc) standard. *IEEE Transactions on circuits and systems for video technology*, 22(12):1649–1668, 2012.
- [66] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 173–191, 2024.
- [67] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In *International Conference on Machine Learning (ICML)*, 2024.
- [68] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A Kozuch, Mor Harchol-Balter, and Gregory R Ganger. Tetrished: global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In *Proceedings of the Eleventh European Conference on Computer Systems*, pages 1–16, 2016.
- [69] Haoyu Wang, Christopher M Pospitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025.
- [70] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, July 2025.
- [71] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.
- [72] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [73] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- [74] Xingyu Xiang, Raj Joshi, Yuhang Liu, Jiayi Yao, Chenxingyu Zhao, Junchen Jiang, Yang Zhou, Eddie Kohler, and Minlan Yu. Shadowserve: Interference-free kv cache fetching for distributed prefix caching. *arXiv preprint arXiv:2509.16857*, 2025.
- [75] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [76] Yi Xiong, Hao Wu, Changxu Shao, Ziqing Wang, Rui Zhang, Yuhong Guo, Junping Zhao, Ke Zhang, and Zhenxuan Pan. Layerkv: Optimizing large language model serving with layer-wise kv cache management. *arXiv preprint arXiv:2410.00428*, 2024.
- [77] Ceyu Xu, Yongji Wu, Xinyu Yang, Beidi Chen, Matthew Lentz, Danyang Zhuo, and Lisa Wu Wills. Llm. 265: Video codecs are secretly tensor codecs. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025.
- [78] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [79] Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.
- [80] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, Jianqun Chen, et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024.
- [81] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.
- [82] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 144–158, 2025.
- [83] Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, et al. Lv-eval: A balanced long-context benchmark with 5 length levels up to 256k. *arXiv preprint arXiv:2402.05136*, 2024.
- [84] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023.
- [85] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems*, 6:196–209, 2024.
- [86] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. 2024.
- [87] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.
- [88] Yufan Zhuang, Xiaodong Yu, Jialian Wu, Ximeng Sun, Ze Wang, Jiang Liu, Yusheng Su, Jingbo Shang, Zicheng Liu, and Emad Barsoum. Self-taught agentic long context understanding. *arXiv preprint arXiv:2502.15920*, 2025.

A Appendix

Appendices are supporting material that has not been peer-reviewed.

A.1 Additional results of image similarity

Fig. 26 plots the PSNR value of slicing the KV cache along different dimensions. The same as SSIM in Fig.11, slicing the KV cache along token dimensions brings the most image similarity.

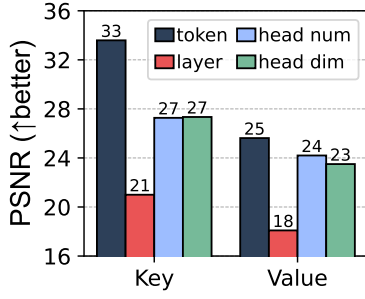


Figure 26: PSNR analysis of slicing KV cache along different dimensions.



Figure 27: Visualization of structural similarity (SSIM) across different dimensions.

Fig. 27 visualizes the different image similarity levels. We add different levels of noise to the original picture, leading to the noised picture having a corresponding SSIM with the original picture. These SSIM values match the SSIM of continuous tensors sliced from the KV cache along different dimensions, e.g., the 0.87 of the token dimension as shown in Fig.11.

A.2 Algorithm of adaptive resolution and corresponding lookup table

Adaptive resolution fetching algorithm is shown in Alg. 1, which aims to minimize pipeline bubbles by synchronizing transmission and decoding stages. Initially, the available bandwidth $\hat{B}_t$ is predicted using historical traces $\mathcal{B}_{t-1}$ (line#1). Next, the algorithm iterates through the set of supported resolutions $\mathcal{R}_{all}$ to evaluate each candidate r (line#4). In each iteration, the transmission latency

Algorithm 1 Adaptive Resolution Selection via Bubble Minimization

Input: Historical Bandwidth $\mathcal{B}_{t-1}$, Current Decompress Pool Load $\mathcal{L}_{pool}$, Profile Table $\mathcal{T}_{prof}$, Support resolution set $\mathcal{R}_{all}$

Output: Optimal Resolution r_{opt}

```

1:  $\hat{B}_t \leftarrow \text{ESTBANDWIDTH}(\mathcal{B}_{t-1})$  ▷ Predict current BW from history
2:  $\delta_{min} \leftarrow \infty$ 
3:  $r_{opt} \leftarrow \text{NULL}$ 
4: for  $r$  in  $\mathcal{R}_{all}$  do ▷ Iterate through all candidate resolutions
5:    $S_r \leftarrow \text{GETVIDEOSize}(r)$ 
6:    $\tau_{trans} \leftarrow S_r / \hat{B}_t$  ▷ Calculate transmission latency
7:    $\tau_{dec}, \tau_{penalty} \leftarrow \text{LOOKUPTABLE}(\mathcal{T}_{prof}, r, \mathcal{L}_{pool})$  ▷ Get decoding latency with switching penalty
8:    $\delta_{bubble} \leftarrow |\tau_{trans} - \tau_{dec} - \tau_{penalty}|$  ▷ Calculate the bubble gap
9:   if  $\delta_{bubble} < \delta_{min}$  then
10:     $\delta_{min} \leftarrow \delta_{bubble}$ 
11:     $r_{opt} \leftarrow r$ 
12:   end if
13: end for
14: return  $r_{opt}$ 

```

τ_{trans} is estimated from the predicted bandwidth and video size S_r (line#5-6). Regarding the decoding latency τ_{dec} , we query the lookup table $\mathcal{T}_{prof}$ based on the current pool load $\mathcal{L}_{pool}$ (line#7). The lookup table across different devices is shown in Tab. 3, Tab. 4 and Tab. 4. Furthermore, we observe that resolution switching within the pool affects the processing efficiency of lower-resolution videos. In addition, to account for resolution switch overheads, a switching penalty $\tau_{penalty}$ whenever the candidate resolution differs from the active resolution in the decompression pool. Finally, the system identifies the optimal resolution r_{opt} that minimizes the bubble δ_{bubble} between transmission and decompression (line#8-11).

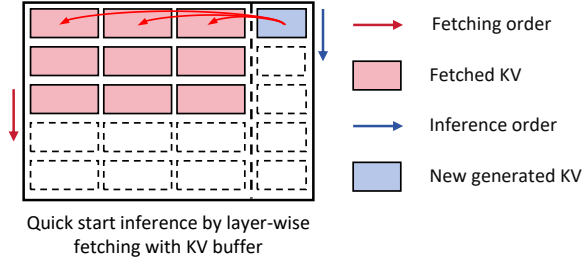
Concurrency	Latency (s)			
	240P	480P	640P	1080P
1	0.21	0.2	0.2	0.19
2	0.22	0.22	0.21	0.19
3	0.29	0.30	0.29	0.26
4	0.32	0.31	0.30	0.30
5	0.46	0.42	0.37	0.35
6	0.52	0.43	0.41	0.40
7	0.62	0.51	0.45	0.43
Penalty	0.08	0.06	0.03	0
Size (MB)	180	205	235	256

Table 3: Lookup table of different resolutions on H20.

Concurrency	Latency (s)			
	240P	480P	640P	1080P
1	0.18	0.175	0.17	0.16
2	0.18	0.178	0.175	0.16
3	0.19	0.183	0.175	0.161
Penalty	0.06	0.06	0.04	0
Size (MB)	180	205	235	256

Table 4: Lookup table of different resolutions on L20.

Concurrency	Latency (s)			
	240P	480P	640P	1080P
1	0.25	0.24	0.231	0.20
2	0.252	0.241	0.235	0.21
3	0.252	0.25	0.24	0.22
4	0.26	0.26	0.25	0.24
5	0.29	0.27	0.27	0.25
Penalty	0.04	0.04	0.03	0
Size (MB)	180	205	235	256

Table 5: Lookup table of different resolutions on A100.**Figure 28: Layer-wise KV fetching with KV buffer.**

A.3 Layer-wise fetching-inference pipeline.

We propose a layer-wise KV fetching pipeline with a KV buffer to achieve a non-blocking execution pipeline as shown in Fig. 28. Following layer-wise pipeline design in Mooncake, KVCodec also pre-allocates GPU memory that entire KV cache required, and fill the fetched remote KV cache into GPU memory in a layer-by-layer manner. To prevent fetching from stalling non-reuse requests, we maintain a KV buffer to track the state of each layer’s KV cache. Fetching requests are added to the running queue only when the pipeline satisfies following non-blocking condition:

$$\sum_{j=1}^k T_{\text{decode}}^{(j)} \leq \sum_{j=1}^{k-1} T_{\text{comp}}^{(j)}, \quad \forall k \in \{L_{\text{buf}} + 1, \dots, L_{\text{total}}\}$$

where $T_{\text{decode}}^{(j)}$ and $T_{\text{comp}}^{(j)}$ denote the decoding and computation time for the j -th layer, L_{buf} and L_{total} denote the buffered layer number and total layer number. This ensures that for every unbuffered layer k , the KV cache data is ready exactly before the GPU completes the computation of layer $k - 1$, thereby preventing any execution stalls. Thanks to the de facto chunked prefill and sequence parallelism techniques, the computation time of each layer can be predicted very precisely.